In [43]:
```
import pandas as pd    #dataframes an ddata
import numpy as np    #numerical operations
import matplotlib.pyplot as plt  #graphs and charts
```

In [3]:
```
#Use Pandas to read the provided CSV file.

df = pd.read_csv(r"C:\Users\Rashid\Downloads\student-scores.csv")
df
```

Out[3]:

	id	first_name	last_name	email	gender	part_time_job
0	1	Paul	Casey	paul.casey.1@gslingacademy.com	male	False
1	2	Danielle	Sandoval	danielle.sandoval.2@gslingacademy.com	female	False
2	3	Tina	Andrews	tina.andrews.3@gslingacademy.com	female	False
3	4	Tara	Clark	tara.clark.4@gslingacademy.com	female	False
4	5	Anthony	Campos	anthony.campos.5@gslingacademy.com	male	False
...	...	...	...	...	...	...
1995	1996	Alan	Reynolds	alan.reynolds.1996@gslingacademy.com	male	False
1996	1997	Thomas	Gilbert	thomas.gilbert.1997@gslingacademy.com	male	False
1997	1998	Madison	Cross	madison.cross.1998@gslingacademy.com	female	False
1998	1999	Brittany	Compton	brittany.compton.1999@gslingacademy.com	female	True
1999	2000	Natalie	Smith	natalie.smith.2000@gslingacademy.com	female	False

2000 rows × 17 columns

In [4]:
```
#Convert the DataFrame to a NumPy array for analysis.
#numpy- used for mean,median,standard deviation and variance.

data_array = df.to_numpy()
data_array
```

Out[4]:
```
array([[1, 'Paul', 'Casey', ..., 63, 80, 87],
       [2, 'Danielle', 'Sandoval', ..., 90, 88, 90],
       [3, 'Tina', 'Andrews', ..., 65, 77, 94],
       ...,
       [1998, 'Madison', 'Cross', ..., 68, 94, 78],
       [1999, 'Brittany', 'Compton', ..., 95, 88, 75],
       [2000, 'Natalie', 'Smith', ..., 83, 93, 100]], dtype=object)
```

In [5]:
```
#Data Cleaning:

df.head()
df.isna().sum()   # 0 means no missing values
df.shape    #no brackets
df.dtypes   #Data types
```

Out[5]:
```
id                            int64
first_name                   object
last_name                    object
email                        object
gender                       object
part_time_job                  bool
absence_days                 int64
extracurricular_activities     bool
weekly_self_study_hours      int64
career_aspiration            object
math_score                   int64
history_score                int64
physics_score                int64
chemistry_score              int64
biology_score                int64
english_score                int64
geography_score              int64
dtype: object
```

In [12]:
```
#EDA:
# mean = np.mean(data_array)
#-cant do because contains text like "names" so cant find average of name so only n

#So selecting subject score columns:

scores = df[["math_score",
             "history_score",
             "physics_score",
             "chemistry_score",
             "biology_score",
             "english_score",
             "geography_score"
             ]]
mean_scores = np.mean(scores, axis=0)   #"axis=0" is mean of each subject sep so ex
mean_scores
```

Out[12]:
```
math_score         83.4520
history_score      80.3320
physics_score      81.3365
chemistry_score    79.9950
biology_score      79.5815
english_score      81.2775
geography_score    80.8880
dtype: float64
```

In [14]:
```
max_scores = np.max(scores, axis =0)
max_scores
```

Out[14]:
```
math_score         100
history_score      100
physics_score      100
chemistry_score    100
biology_score      100
english_score       99
geography_score    100
dtype: int64
```

In [15]:
```
min_scores = np.min(scores, axis=0)
min_scores
```

Out[15]:
```
math_score         40
history_score      50
physics_score      50
chemistry_score    50
biology_score      30
english_score      50
geography_score    60
dtype: int64
```

In [17]:
```
standard_deviation = np.std(scores, axis=0)
standard_deviation
```

Out[17]:
```
math_score         13.221600
history_score      12.732862
physics_score      12.536318
chemistry_score    12.774701
biology_score      13.718759
english_score      12.024080
geography_score    11.634795
dtype: float64
```

In [19]:
```
variance = np.var(scores, axis=0)
variance
```

Out[19]:
```
math_score         174.810696
history_score      162.125776
physics_score      157.159268
chemistry_score    163.192975
biology_score      188.204358
english_score      144.578494
geography_score    135.368456
dtype: float64
```

In [30]:
```
#to find highest variance scores in numpy:
#first find highest position
highest_var_position = np.argmax(variance)
highest_var_position
```

Out[30]:
```
4
```

In [35]:
```
# then Use that possiiton to find subject name
highest_variance_subjects = scores.columns[highest_var_position]
highest_variance_subjects
```

```
subject with highest variance: biology_score
Highest variance score: 4
```

In [36]:
```
highest_variance_score = variance.iloc[highest_var_position]
highest_variance_score
```

Out[36]:
```
188.20435775000163
```

In [37]:
```
print("subject with highest variance:", highest_variance_subjects)

print("Highest variance score:", highest_variance_score)
```

```
subject with highest variance: biology_score
Highest variance score: 188.20435775000163
```

In [20]:
```
median = np.median(scores, axis=0)
median
```

Out[20]:
```
array([87., 82., 83., 81., 81., 83., 81.])
```

In [25]:
```
#Tried using -   mode = np.mode(scores, axis=0) but numpy doesnt have mode or range

mode = scores.mode()
mode
```

Out[25]:

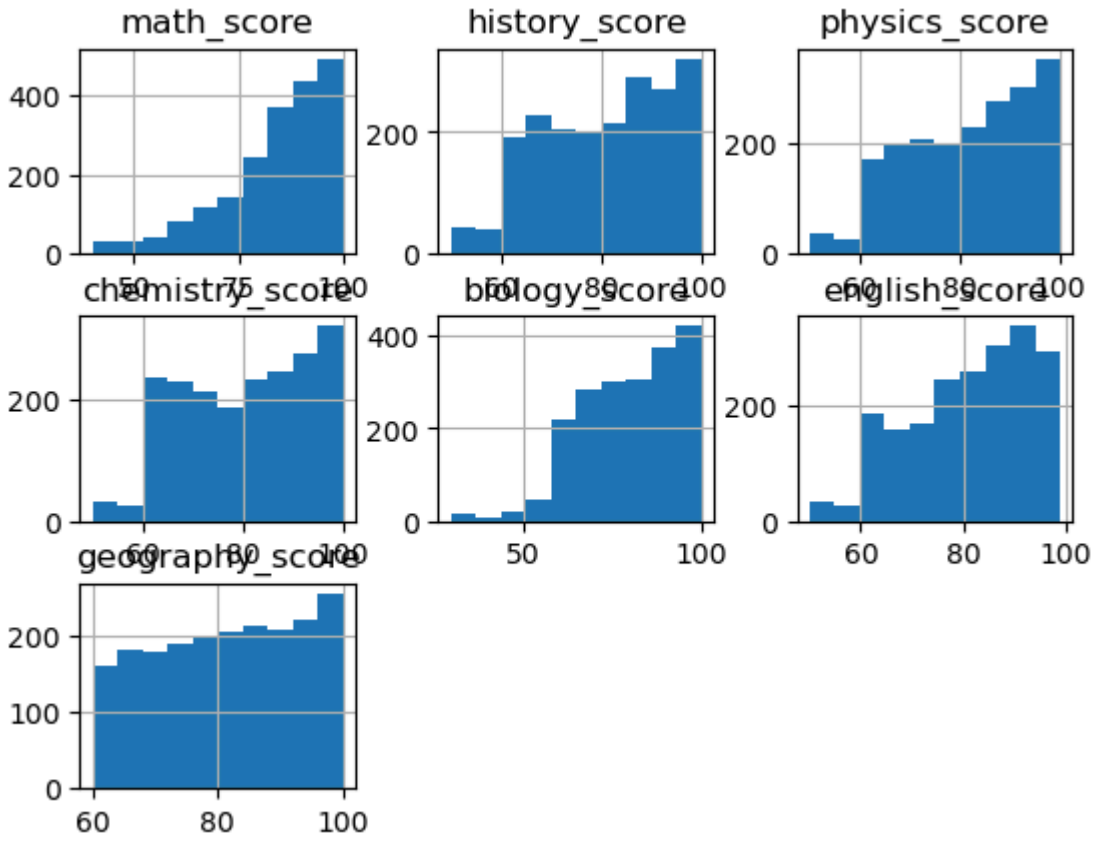
	math_score	history_score	physics_score	chemistry_score	biology_score	english_score	geograp
0	99.0	88.0	96.0	94	100.0	90.0	
1	NaN	NaN	NaN	96	NaN	NaN	

In [27]:
```
# tried using numpy but doesnt have range function range = np.range(scores, axis=0)
range = max_scores - min_scores
range
```

Out[27]:
```
math_score         60
history_score      50
physics_score      50
chemistry_score    50
biology_score      70
english_score      49
geography_score    40
dtype: int64
```

In [44]:
```
#Task 5:Histograms-Data visualisations:
scores = df[["math_score",
             "history_score",
             "physics_score",
             "chemistry_score",
             "biology_score",
             "english_score",
             "geography_score"
             ]]

scores.hist()   #histograms for each subject
plt.show()
```



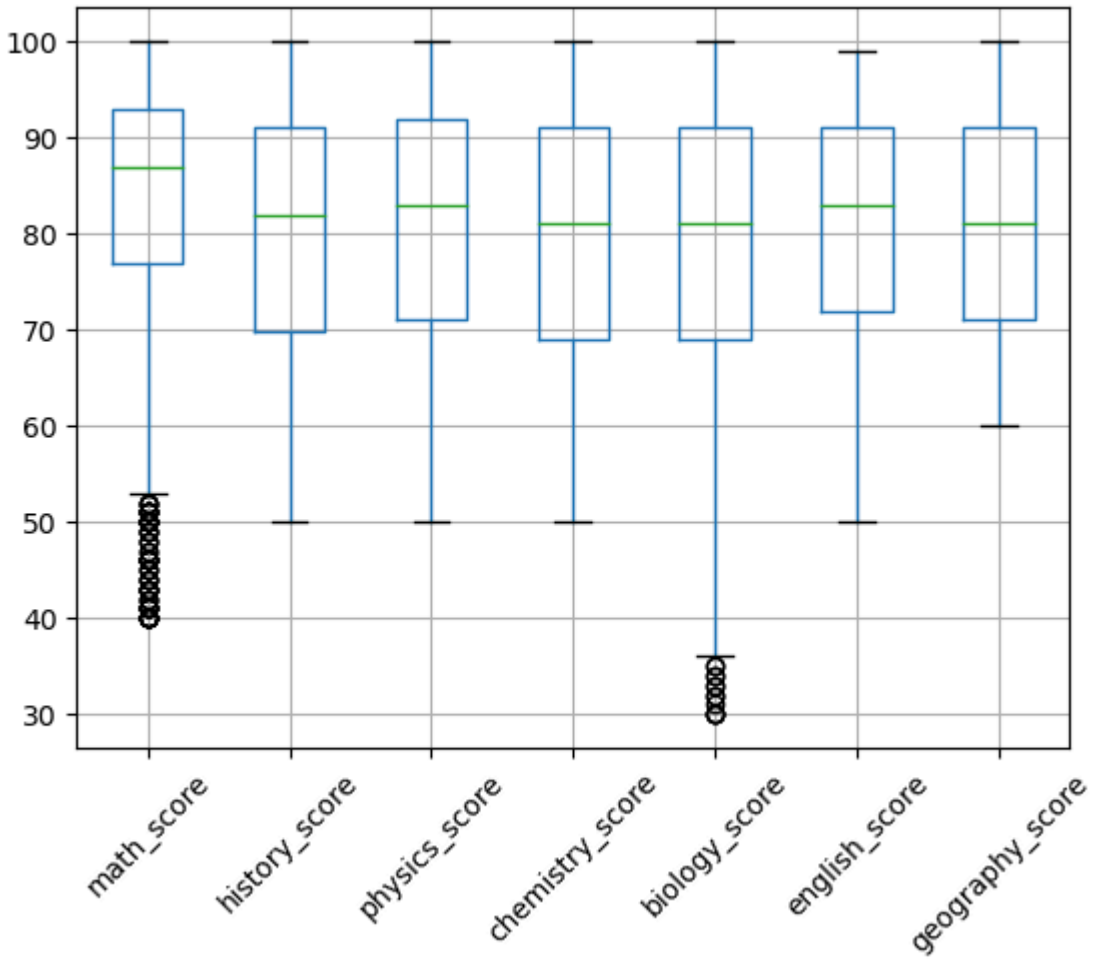
In [39]:
```
#Box plots for each subject scores:
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[39], line 1
----> 1 plt.show()

File ~\anaconda3\Lib\site-packages\matplotlib\_api\__init__.py:226, in caching_mod
ule_getattr.<locals>.__getattr__(name)
    224 if name in props:
    225     return props[name].__get__(instance)
--> 226 raise AttributeError(
    227     f"module {cls.__module__!r} has no attribute {name!r}")

AttributeError: module 'matplotlib' has no attribute 'show'
```

In [47]:
```
scores.boxplot(rot=45)
plt.show()
```



In [48]:
```
#Task 6: P1: Correlation Analysis:

# Find the correlation between the different subjects
correlation = np.corrcoef(scores, rowvar=False)   #"rowvar=false" means subjects ar
correlation
```

Out[48]:
```
array([[1.        , 0.14724747, 0.11571874, 0.12713149, 0.08129806,
        0.13483078, 0.04967233],
       [0.14724747, 1.        , 0.04847843, 0.12149786, 0.08850197,
        0.14719288, 0.06575135],
       [0.11571874, 0.04847843, 1.        , 0.1261628 , 0.13227971,
        0.0543137 , 0.10312592],
       [0.12713149, 0.12149786, 0.1261628 , 1.        , 0.11999168,
        0.06834058, 0.06543008],
       [0.08129806, 0.08850197, 0.13227971, 0.11999168, 1.        ,
        0.07422691, 0.10652591],
       [0.13483078, 0.14719288, 0.0543137 , 0.06834058, 0.07422691,
        1.        , 0.07224979],
       [0.04967233, 0.06575135, 0.10312592, 0.06543008, 0.10652591,
        0.07224979, 1.        ]])
```

In [50]:
```
#Task 6:P2: Outliers:

q1 = scores.quantile(0.25)
q3 =scores.quantile(0.75)
iqr = q3-q1
#Find potential outliers
outliers = (scores < (q1 - 1.5 * iqr)) | (scores > (q3 + 1.5 * iqr))
#Count no. of outliers for each subject
outliers.sum()
```

Out[50]:
```
math_score         70
history_score       0
physics_score       0
chemistry_score     0
biology_score      13
english_score       0
geography_score     0
dtype: int64
```

In [ ]: